Dealing with Missing Values

Study notes for the Home Credit loan default project (load-default-predictor)

1. First: why is it missing?

The right method depends on why the value is missing. Statisticians use three categories:

- **MCAR (missing completely at random):** the gap has nothing to do with the data, such as a sensor glitch. This is rare, and it is the only case where simply dropping rows is safe.
- **MAR (missing at random):** the gap depends on *other columns you can see*. For example, OWN_CAR_AGE is missing when FLAG_OWN_CAR is N. You can use the other columns to fill or interpret it.
- **MNAR (missing not at random):** the gap depends on the missing value itself or on something you cannot see. For example, people with high debt skip an income field. The missingness itself is informative here.

In practice a fourth case matters more than the textbook ones: **not applicable**. The value does not exist, as with the age of a car the person does not have, or an employer for a pensioner. Imputing a number here invents information.

You can test this by comparing default rates between missing and present rows. If the rates differ, the gap carries information.

2. The methods

A. Deletion

- **Drop rows (listwise deletion).** Easy, but you lose data and introduce bias unless the data is MCAR. With 41 columns more than 50% missing, dropping rows would remove most of the dataset. At prediction time you also cannot drop a customer you need to score.
- **Drop columns.** Reasonable for columns that are nearly empty *and* uninformative. Check the target relationship first. The housing and bureau columns have signal even though they are mostly missing.

B. Simple imputation (fill with a statistic)

- **Mean:** fine for roughly symmetric numbers, but pulled by outliers and it shrinks the variance.
- **Median:** the usual default for skewed numbers such as income.
- **Mode / most frequent:** for categoricals.
- **Constant:** such as 0, or the category "Missing".

Simple and fast. The drawback is that every filled row gets the same value, which weakens relationships between features and can hide the signal the gap carried.

C. Missing-indicator flags

Add a 0/1 column saying "this value was missing", and impute separately. This keeps the information in the gap while the model still gets a complete column. In scikit-learn, SimpleImputer(add_indicator=True) does it. For block-structured data like this dataset, one flag per block is enough.

D. Out-of-range sentinel

Fill with something like -999 so a tree model can separate "missing" from real values. It works with trees, but it is dangerous for linear and distance-based models, where -999 is treated as a real number.

E. Group-wise imputation

Fill using the statistic of a related subgroup, such as median income by occupation or education. It respects MAR structure and is usually better than one global value. Compute the statistic on training data only.

F. Model-based imputation

- **KNN imputer:** fill from similar rows. Slow on 300k rows with 100+ columns, and sensitive to scaling.
- **Iterative (MICE-style) imputer:** predicts each column from the others in rounds. Good when columns are related, but slower and more complex, and it can leak if done wrongly.
- **A dedicated model** for a single important column.

These can be more accurate, but they cost time and complexity, and they often do not beat simple imputation plus flags in a predictive model.

G. Let the model handle it

Gradient-boosted trees (LightGBM, XGBoost, scikit-learn's HistGradientBoosting) accept NaN directly and learn which side of a split missing values should go. This preserves the signal in the gap with no imputation at all. Linear models, SVMs and most neural networks need complete data.

H. Domain rules

Sometimes the correct fill comes from understanding the column. "No car, so car age is not applicable" might be a flag plus a neutral fill. "No bureau record" might mean a genuine zero inquiries. This is often the best option, and it needs the data dictionary and the checks you have been doing.

3. The traps that matter more than the method

- **Data leakage.** Fit every imputer on the *training* data only, then apply it to validation and test data. If you compute medians on the whole dataset first, test information leaks into training. Put imputation inside a scikit-learn Pipeline / ColumnTransformer so cross-validation refits it on each fold.
- **Hidden missing values.** Convert placeholders (365243, XNA) to NaN *before* imputing, or the imputer treats them as real values. Do this in the pipeline too, so training and test are handled identically.
- **Imputing before understanding.** Imputing can erase a signal you had found, which is why flags matter.
- **Different treatment in training and serving.** Whatever you do must also work on a single new applicant at prediction time.
- **The same fact repeated.** Imputing 40 housing columns independently invents a plausible-looking house for people who gave no housing data.

4. How this maps to the project data

These are options to weigh, not final answers. Choose and justify them yourself.

Block	What was found	Options to weigh
Housing (47 columns)	Partly block-structured; missing goes with higher default (8.6% vs 6.9%)	Flag for "housing data missing" plus native-NaN trees or median fill; maybe drop redundant _MODE / _MEDI duplicates
Credit bureau (6)	Identical missingness; missing goes with higher default (10.3% vs 7.7%)	Flag, plus 0 or median fill (depends on what "no record" means)
Social circle (4)	Identical missingness; tiny group (1,021 rows), lower default	Flag or just fill; low impact
EXT_SOURCE_*	Independent gaps; strong predictors	Individual flags plus median fill, or native-NaN trees; do not group
OWN_CAR_AGE	Missing most likely means no car (verify against FLAG_OWN_CAR)	Flag from FLAG_OWN_CAR plus a neutral fill
DAYS_EMPLOYED / ORGANIZATION_TYPE	Placeholder on the same 55,374 rows; 99.96% are pensioners; default 5.4% vs 8.7%	Convert 365243 to NaN, keep an indicator or rely on income type, treat XNA as its own category

5. Quick reference

Method	Keeps the signal in the gap?	Needs care with
Drop rows	No	Bias, lost data, cannot use at prediction time
Drop column	No	Dropping informative columns
Mean / median / mode	No (unless flagged)	Fit on training data only
Median + indicator flag	Yes	Number of flag columns
Sentinel value	Yes (trees only)	Linear and distance-based models
Group-wise fill	Partly	Small groups, leakage
KNN / iterative	Partly	Speed, scaling, leakage
Native NaN (GBMs)	Yes	Model choice (not for linear models)
Domain rule	Depends	Needs the data dictionary

6. Decisions to make next

- Pick the first model family. Gradient-boosted trees can use native NaN plus a few flags. Logistic regression as a baseline needs real imputation. Many projects do both.
- For each block write one row: what you found, the chosen treatment, why, and what you would check if it hurt performance.
- Do all the filling inside a Pipeline, after the train/test split.